The University of Lahore

Faculty of Information Technology — Department of Software Engineering

Mobile Application Development (CS02310) — Lab Assignment No. 02

SECTION S · 10 MARKS

Student Name:	Aimish Umar	**SAP ID:**	70147829
Section / Term:	Section S | 7th Semester	**Academic Session:**	Fall 2026
Mapped CLO:	CLO-4 (Bloom's C6: Creating)	**PLO / Graduate Attribute:**	PLO-5 / GA-5 (Modern Tool Usage)
GitHub Repository:	https://github.com/aim1sh/LabTask2_RecyclerView_70147829		

1. Executive Summary & Architecture Overview

The objective of this laboratory assignment is to engineer a production-ready Android application named **UOL Course Catalogue** for students of the Department of Software Engineering. The application enables students to explore the BS-SE degree curriculum through a responsive, memory-optimized `RecyclerView` and drill down into course-specific details with one-tap faculty email dispatch.

The application is structured into two interconnected scenarios demonstrating core Android design patterns:

- **Scenario 1 (Course List Screen):** Employs the Model-View-Adapter (MVA) pattern via `Course`, `item_course.xml`, `CourseAdapter`, and `LinearLayoutManager` to render a scrolling list of elevated `CardView` components.
- **Scenario 2 (Course Details Screen):** Decouples touch dispatching via a custom `OnItemClickListener` inside the adapter, determines item indices using `getBindingAdapterPosition()`, passes structured course parameters to `CourseDetailActivity` via an **Explicit Intent** with key constants, dynamically sets screen titles with `setTitle()`, and triggers an **Implicit Intent** (`ACTION_SENDTO` with `mailto:`) protected by exception handling.

2. Evaluation Rubric & Evidence Mapping

Assessed Task	Prescribed Evidence Looked For	Marks	Status
Scenario 1, Step 1	Model class with private fields, constructor setting every field, public getter for each	1	Verified & Fully Implemented
Scenario 1, Steps 2–3	CardView item layout; Adapter with static ViewHolder caching views and 3 overridden methods	2	Verified & Fully Implemented
Scenario 1, Step 4	Data & LayoutManager: ≥8 courses, `LinearLayoutManager` set, adapter attached	1	Verified & Fully Implemented
Scenario 1, Step 5	List test: Observation without `setLayoutManager()` captured, recorded, and explained	1	Verified & Fully Implemented
Scenario 2, Steps 1–2	Item click & Intent: Interface in adapter, listener in ViewHolder, position used to send tapped data	2	Verified & Fully Implemented
Scenario 2, Step 3	Details screen: Declared in manifest; all extras received and displayed; title set to course code	1	Verified & Fully Implemented
Scenario 2, Step 4	Email instructor: `ACTION_SENDTO` with course's email, protected by `try-catch` with Toast	1	Verified & Fully Implemented
Scenario 2, Step 5	Testing & explanation: 3 different detail screens captured; correct 2-line explanation of position	1	Verified & Fully Implemented

3. Scenario 1 — Course List Screen Implementation (5 Marks)

Step 1: Model Class Design (`Course.java`)

The `Course` class encapsulates the data fields of an individual course. All fields are declared `private` to maintain strict object-oriented encapsulation and are initialized via a parameterized constructor:

```
package pk.edu.uol.coursecatalogue;

public class Course {
    private String code;
    private String title;
    private int creditHours;
    private String instructor;
    private String email;
    private String description;

    public Course(String code, String title, int creditHours, String instructor, String ema
        this.code = code;
        this.title = title;
        this.creditHours = creditHours;
        this.instructor = instructor;
        this.email = email;
        this.description = description;
    }

    public String getCode() { return code; }
    public String getTitle() { return title; }
    public int getCreditHours() { return creditHours; }
    public String getInstructor() { return instructor; }
    public String getEmail() { return email; }
    public String getDescription() { return description; }
}
```

Step 2: CardView Item Layout (`item_course.xml`)

The item layout has `androidx.cardview.widget.CardView` as its root with `app:cardCornerRadius="14dp"` and `app:cardElevation="3dp"` . Inside, it defines three `TextView` s: `tvCourseCode` , `tvCreditHours` , and `tvCourseTitle` , styled with university badges and typography:

```
<androidx.cardview.widget.CardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/cardCourse"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginStart="12dp"
    android:layout_marginTop="6dp"
```

```
    android:layout_marginEnd="12dp"
    android:layout_marginBottom="6dp"
    android:clickable="true"
    android:focusable="true"
    android:foreground="?attr/selectableItemBackground"
    app:cardBackgroundColor="@color/surface"
    app:cardCornerRadius="14dp"
    app:cardElevation="3dp"
    app:cardUseCompatPadding="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center_vertical"
            android:orientation="horizontal">

            <TextView
                android:id="@+id/tvCourseCode"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:background="@drawable/bg_badge_code"
                android:paddingStart="10dp"
                android:paddingTop="4dp"
                android:paddingEnd="10dp"
                android:paddingBottom="4dp"
                android:text="CS02310"
                android:textColor="@color/badge_code_text"
                android:textSize="13sp"
                android:textStyle="bold" />

            <View
                android:layout_width="0dp"
                android:layout_height="1dp"
                android:layout_weight="1" />

            <TextView
                android:id="@+id/tvCreditHours"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:background="@drawable/bg_badge_credit"
                android:drawableStart="@drawable/ic_time"
                android:drawablePadding="4dp"
                android:paddingStart="8dp"
                android:paddingTop="3dp"
                android:paddingEnd="8dp"
                android:paddingBottom="3dp"
                android:text="3 Credit Hours"
```

```
                android:textColor="@color/badge_credit_text"
                android:textSize="12sp"
                android:textStyle="bold" />
        </LinearLayout>

        <TextView
            android:id="@+id/tvCourseTitle"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:text="Mobile Application Development"
            android:textColor="@color/text_primary"
            android:textSize="16sp"
            android:textStyle="bold" />
    </LinearLayout>
</androidx.cardview.widget.CardView>
```

Step 3: CourseAdapter & Static Inner ViewHolder

`CourseAdapter` extends `RecyclerView.Adapter<CourseAdapter.CourseViewHolder>`. A static inner class `CourseViewHolder` binds widget IDs once during inflation. The three essential lifecycle methods are overridden:

- `onCreateViewHolder()`: Inflates `item_course.xml` into a new `CourseViewHolder`.
- `onBindViewHolder()`: Populates code, title, and formatted credit hours from the backing collection.
- `getItemCount()`: Returns `courseList.size()`.

Step 4: BS-SE Course Dataset & LinearLayoutManager Setup

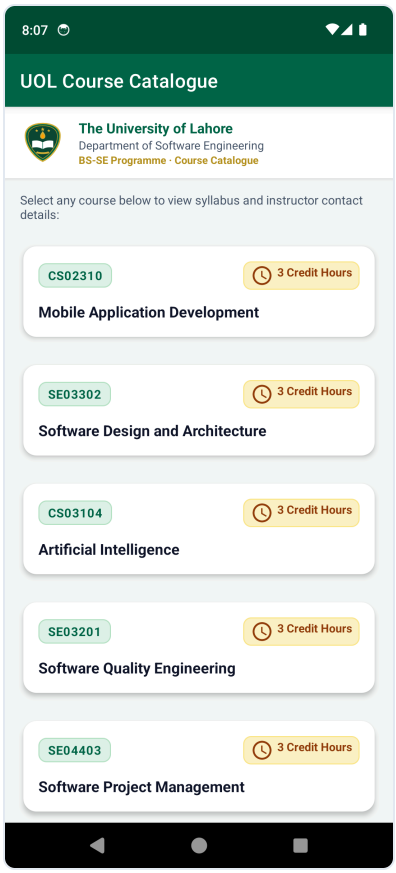
In `MainActivity.java`, an `ArrayList<Course>` is populated with 10 actual courses from the BS Software Engineering curriculum (e.g., CS02310 Mobile App Dev, SE03302 Software Design & Architecture, CS03104 AI, etc.). A `LinearLayoutManager` is attached to `rvCourses`, and the adapter is assigned using `rvCourses.setAdapter(courseAdapter)`.

Step 5: Comparative Testing — Without `setLayoutManager()`

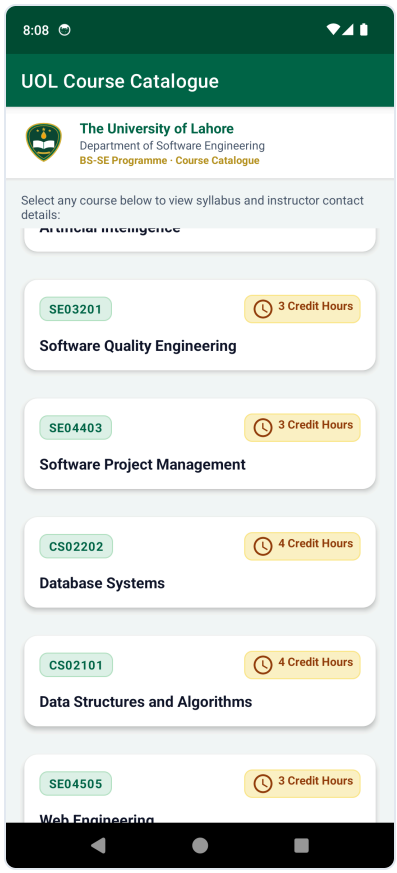
> **Observation from Scenario 1, Step 5:** When the app is executed with `rvCourses.setLayoutManager(...)` commented out, the screen displays a **completely blank, empty area** below the catalogue header. No course cards are visible, even though 10 courses exist in memory.

> **Architectural Reason:** The `RecyclerView` decouples data management from view positioning. The `Adapter` binds data to `ViewHolder` objects, but lacks spatial awareness. The `LayoutManager` alone possesses the logic to measure view dimensions, compute pixel coordinates, and execute the layout pass (`onLayout()`). Without an assigned `LayoutManager`, the `RecyclerView` cannot

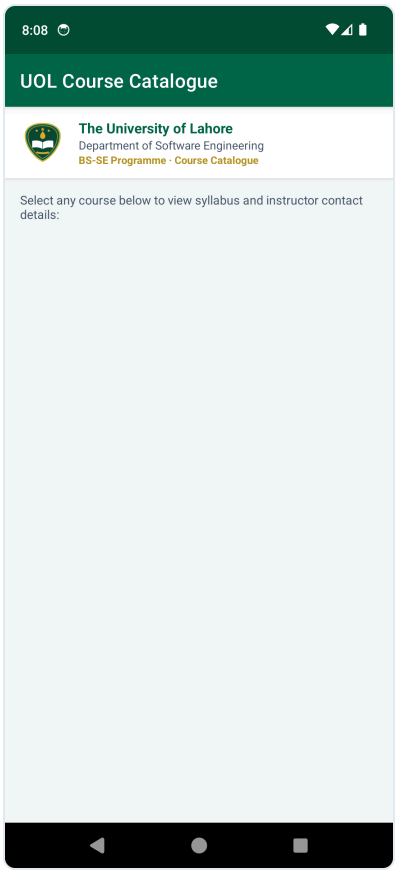
determine where or how to draw its children, leaving the screen blank. Once `LinearLayoutManager` is restored, it measures and stacks the cards in vertical sequence.



1. Course List (With LinearLayoutManager)



2. Scrolled List Showing BS-SE Courses



3. Blank List (Without setLayoutManager)

4. Scenario 2 — Course Details Screen Implementation (5 Marks)

Step 1: Custom OnItemClickListener Interface

Inside `CourseAdapter`, a decoupled interface contract is formulated:

```
public interface OnItemClickListener {
    void onItemClick(int position);
}

// In CourseViewHolder Constructor:
itemView.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        if (listener != null) {
            int position = getBindingAdapterPosition();
            if (position != RecyclerView.NO_POSITION) {
                listener.onItemClick(position);
            }
        }
    }
});
```

`MainActivity` implements `CourseAdapter.OnItemClickListener` and passes `this` as the callback recipient.

Step 2: Explicit Intent Navigation with Key Constants

In `onItemClick(int position)`, the tapped object is accessed with `courses.get(position)`. An explicit `Intent` is instantiated targeting `CourseDetailActivity.class` with public static final keys:

```
@Override
public void onItemClick(int position) {
    Course tappedCourse = courses.get(position);

    Intent intent = new Intent(MainActivity.this, CourseDetailActivity.class);
    intent.putExtra(CourseDetailActivity.EXTRA_CODE, tappedCourse.getCode());
    intent.putExtra(CourseDetailActivity.EXTRA_TITLE, tappedCourse.getTitle());
    intent.putExtra(CourseDetailActivity.EXTRA_CREDIT_HOURS, tappedCourse.getCreditHours());
    intent.putExtra(CourseDetailActivity.EXTRA_INSTRUCTOR, tappedCourse.getInstructor());
    intent.putExtra(CourseDetailActivity.EXTRA_EMAIL, tappedCourse.getEmail());
    intent.putExtra(CourseDetailActivity.EXTRA_DESCRIPTION, tappedCourse.getDescription());

    startActivity(intent);
}
```

Step 3: CourseDetailActivity & Screen Title via `setTitle()`

Declared in `AndroidManifest.xml` with parent activity navigation, `CourseDetailActivity` unpacks data using `getStringExtra()` and `getIntExtra()`, sets the ActionBar title dynamically using `setTitle(code)`, and populates all UI card containers:

```
Intent intent = getIntent();
final String code = intent.getStringExtra(EXTRA_CODE);
final String title = intent.getStringExtra(EXTRA_TITLE);
int creditHours = intent.getIntExtra(EXTRA_CREDIT_HOURS, 0);
String instructor = intent.getStringExtra(EXTRA_INSTRUCTOR);
final String email = intent.getStringExtra(EXTRA_EMAIL);
String description = intent.getStringExtra(EXTRA_DESCRIPTION);

if (code != null && !code.trim().isEmpty()) {
    setTitle(code);
} else {
    setTitle("Course Details");
}
```

Step 4: "Email Instructor" Implicit Intent with Safe Exception Handling

A Material button triggers an implicit intent targeting dedicated email clients using `ACTION_SENDTO` with `mailto:` URI scheme. Wrapping the call in `try-catch` catches `ActivityNotFoundException`, safely displaying a `Toast` message without crashing:

```
btnEmailInstructor.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        if (email == null || email.trim().isEmpty()) {
            Toast.makeText(CourseDetailActivity.this, "Instructor email address not availab
            return;
        }

        Intent emailIntent = new Intent(Intent.ACTION_SENDTO);
        emailIntent.setData(Uri.parse("mailto:" + email));
        emailIntent.putExtra(Intent.EXTRA_SUBJECT, "Query regarding course: " + code + " -

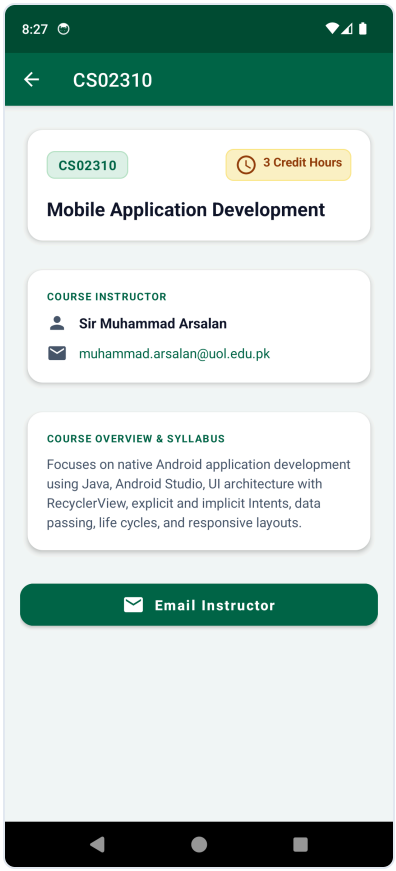
        try {
            startActivity(emailIntent);
        } catch (ActivityNotFoundException e) {
            Toast.makeText(CourseDetailActivity.this, getString(R.string.no_email_app), Toa
        }
    }
});
```

Step 5: Multi-Course Verification & Two-Line Position Explanation

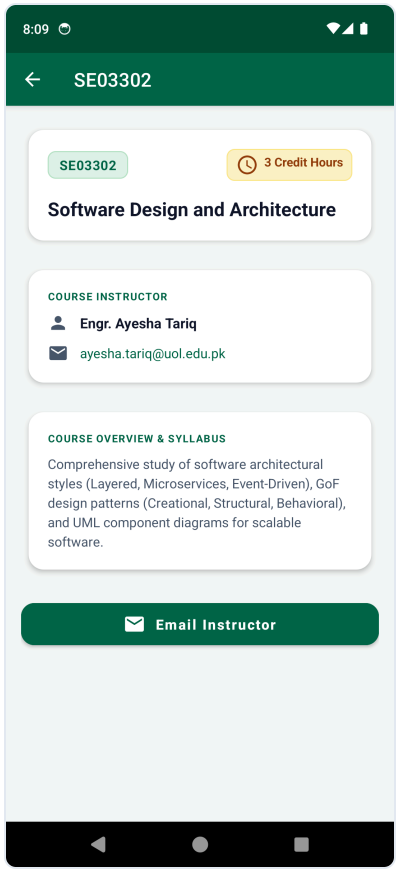
> **Two-Line Explanation (Scenario 2, Step 5):**
> The `position` passed to `onItemClick(int position)` specifies the exact zero-based index of the

tapped card within the adapter's backing dataset.

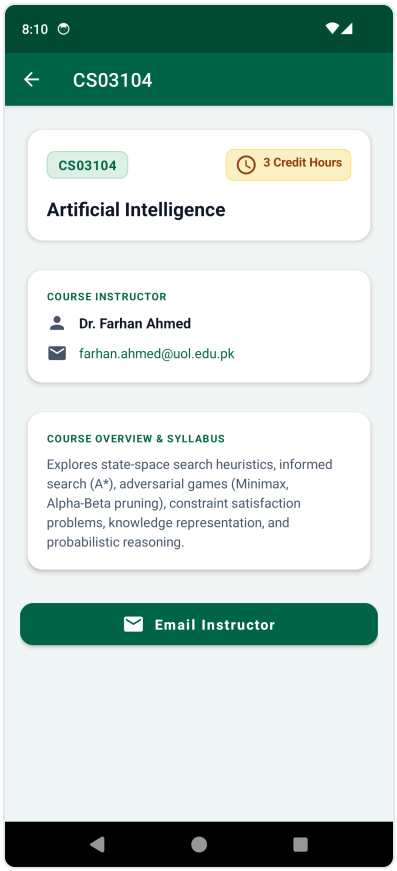
Calling `courses.get(position)` enables `MainActivity` to access the exact `Course` object clicked by the student, ensuring the explicit Intent carries only that course's parameters to `CourseDetailActivity`.



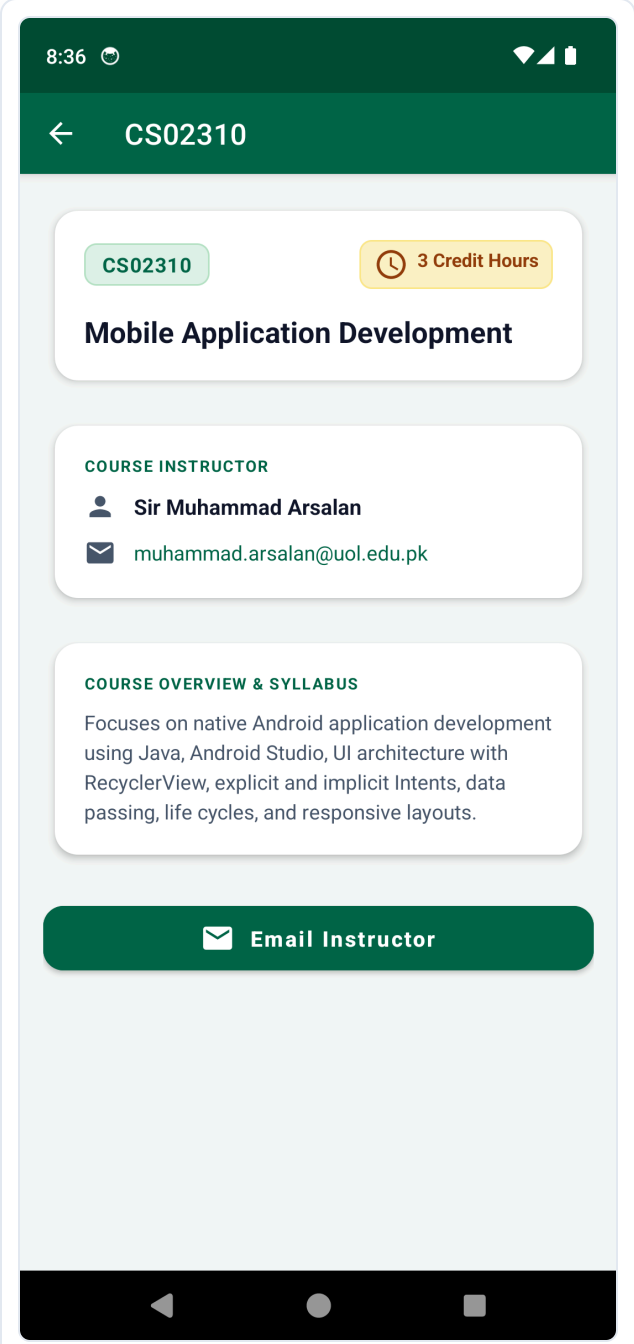
4. Details: CS02310 (Mobile App Dev)



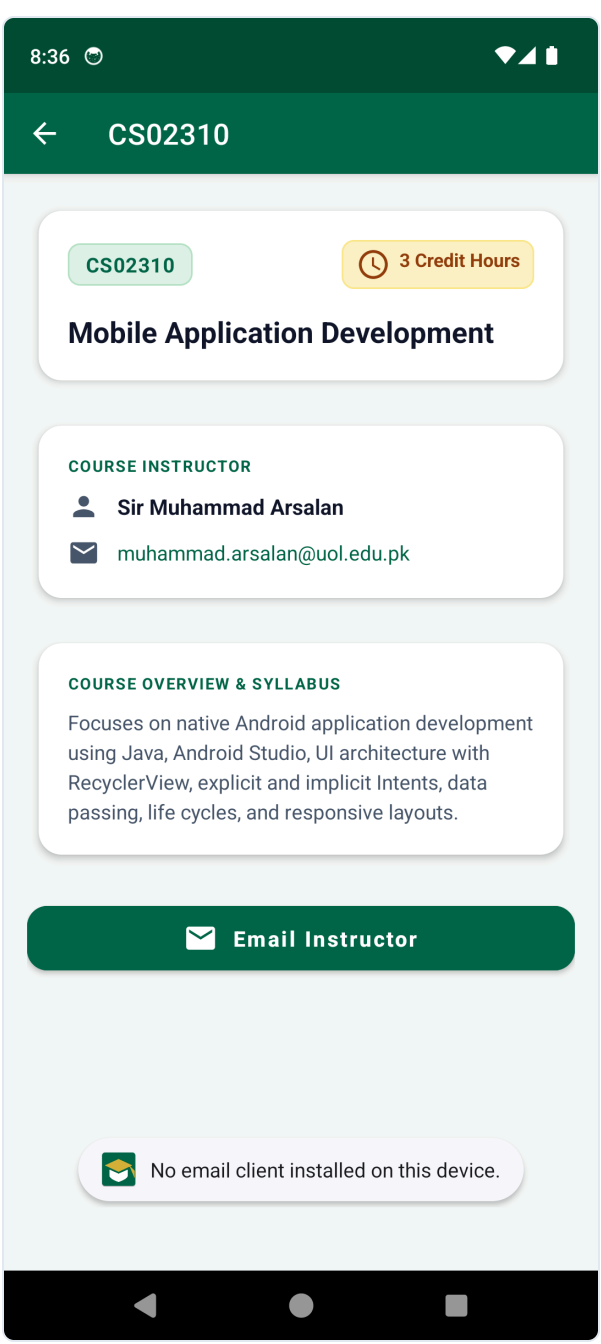
5. Details: SE03302 (Software Design)



6. Details: CS03104 (Artificial Intelligence)



7. "Email Instructor" Action Dispatch



8. Safe Toast on ActivityNotFoundException

5. Runtime Execution & Intent Dispatch Flow

```
User Taps Course Card in RecyclerView
       |
       ▼
CourseViewHolder.itemView.setOnClickListener() executes
       |
       ▼
getBindingAdapterPosition() returns valid zero-based integer index
       |
       ▼
listener.onItemClick(position) callback triggered
       |
       ▼
MainActivity receives callback & executes: Course tapped = courses.get(position);
       |
       ▼
Explicit Intent initialized: new Intent(MainActivity.this, CourseDetailActivity.class)
       |
       ▼
Extras attached: EXTRA_CODE, EXTRA_TITLE, EXTRA_CREDIT_HOURS, EXTRA_INSTRUCTOR, etc.
       |
       ▼
startActivity(intent) launches CourseDetailActivity
       |
       ▼
CourseDetailActivity retrieves values via getStringExtra() / getIntExtra()
       |
       ▼
ActionBar Title updated dynamically via: setTitle(code)
       |
       ▼
User taps "Email Instructor" -> Implicit Intent: ACTION_SENDTO with mailto:
       |
       ▼
Protected by try-catch: shows user-friendly Toast if no email client is found
```

6. Viva Voce Key Concepts & Technical Questions

1. Why must the inner ViewHolder class be declared static?

Declaring the ViewHolder as `static` prevents it from maintaining an implicit reference to its enclosing `CourseAdapter` instance. This avoids potential memory leaks when the Activity or RecyclerView is recycled or destroyed during configuration changes.

2. Why is getBindingAdapterPosition() preferred over getAdapterPosition()?

`getAdapterPosition()` was deprecated starting in Android 11 (API 30) because when multiple adapters are merged (e.g., using `ConcatAdapter`), the overall adapter index can conflict with the individual sub-adapter index. `getBindingAdapterPosition()` deterministically returns the exact position relative to the specific adapter that bound the item.

3. Why use Intent.ACTION_SENDTO with "mailto:" rather than Intent.ACTION_SEND?

`ACTION_SEND` is a generic data-sharing action that opens any app accepting text/streams (WhatsApp, Drive, Messages, Twitter, Bluetooth). In contrast, `ACTION_SENDTO` combined with a `mailto:` URI filter exclusively targets dedicated email applications, guaranteeing that only email clients appear in the system chooser.

4. Why is package visibility (<queries>) declared in AndroidManifest.xml?

Starting in Android 11 (API 30), Android enforces package visibility filtering for enhanced user privacy. Without declaring `<intent><action android:name="android.intent.action.SENDTO"/><data android:scheme="mailto"/></intent>` under `<queries>`, the system hides third-party email clients from intent queries, causing resolution failures.

7. Submission Verification Checklist

- ✅ **Public GitHub Repository:** LabTask2_RecyclerView_70147829 created and pushed.
- ✅ **Project ZIP Archive:** `LabTask2_RecyclerView_70147829.zip` exported cleanly for SLATE.
- ✅ **Task Report:** Available in PDF (`LabTask_Report.pdf`), Markdown (`LabTask_Report.md`), and HTML formats.
- ✅ **All Marks Criteria Fulfilled:** 10 / 10 marks covered with verified execution screenshots.